fit@hcmus
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG - HCM
KHOA CÔNG NGHỆ THÔNG TIN

Seminar — Đề tài 9

ANSIBLE CONFIGURATION MANAGEMENT

Agentless · Idempotent · Infrastructure as Code

Giảng viên hướng dẫn: Lê Hà Minh
Sinh viên thực hiện:
Lê Quốc Hưng, Nguyễn Đức Duy, Nguyễn Thành Dự, Âu Bảo Trân, Mai Thúc Hải Đăng

01 Bài toán & sự ra đời

02 Kiến trúc Agentless

03 Các thành phần cơ bản

04 Idempotence

05 Module chuyên dụng vs Shell

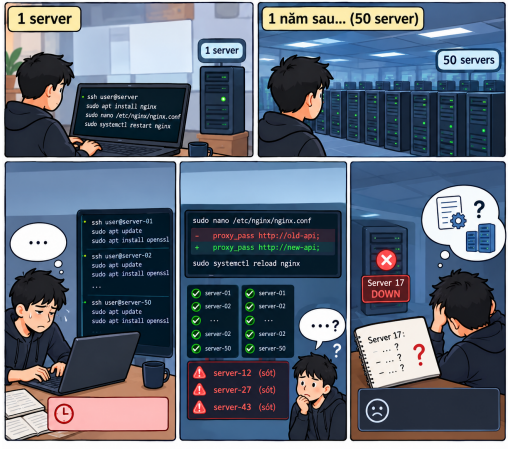
06 Ansible Vault

07 Demo

08 Tổng kết & Hỏi đáp

Bài toán: từ 1 server lên 50 server

1 server → **50 server**, nhưng vẫn SSH vào từng máy và sửa bằng tay



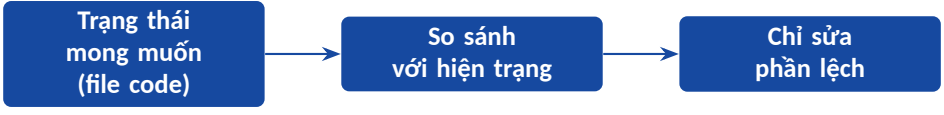
Thử giải bằng tay và bằng script bash

	Làm tay	Script bash
Nhiều máy	Rất tốn công, dễ sót	Tự viết vòng lặp, tự xử lý lỗi
Chạy lại	Gõ lại từ đầu	Lỗi ở bước đã làm rồi (`mkdir: File exists`, `useradd: already exists`)
Biết máy đang ra sao	Không	Không
Tài liệu hoá	Nằm trong đầu người làm	Script dài, khó đọc

> **Hậu quả — Configuration drift:** mỗi máy lệch đi một kiểu sau nhiều lần sửa tay.

Hướng giải: Configuration Management & IaC

- Không viết *"làm thế nào"* mà **mô tả trạng thái mong muốn**: *"nginx phải được cài, config phải có nội dung X, service phải đang chạy"*.
- Công cụ tự so sánh với hiện trạng và **chỉ sửa phần lệch** → chạy lại bao nhiêu lần cũng an toàn (**idempotent**).
- **Infrastructure as Code**: cấu hình là file code → lưu Git, review, dựng lại server từ file.



Thế hệ công cụ đầu tiên

Công cụ	Năm	Agent	Mô hình	Ngôn ngữ
CFEngine	1993	Có agent	Pull	DSL riêng
puppet	2005	Có agent	Pull	Puppet DSL
CHEF	2009	Có agent (Chef Infra Client)	Pull	Ruby DSL
SALTSTACK	2011	`salt-minion` (Salt SSH thì agentless)	Push / lịch chạy	YAML + Jinja2

Giải quyết được bài toán, nhưng phải:

- Cài và cập nhật **agent trên từng máy**, dựng **server trung tâm**, quản lý **chứng chỉ/key**.
- Học **DSL hoặc Ruby** trước khi viết được cấu hình đầu tiên.

Ansible ra đời (2012)



01 Bài toán & sự ra đời

02 Kiến trúc Agentless

03 Các thành phần cơ bản

04 Idempotence

05 Module chuyên dụng vs Shell

06 Ansible Vault

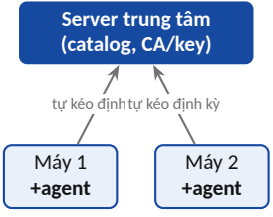
07 Demo

08 Tổng kết & Hỏi đáp

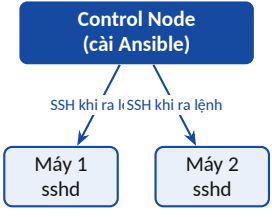
Agentless là gì?

- **Agent** = chương trình chạy thường trực trên mỗi máy được quản lý, liên lạc với server trung tâm để nhận cấu hình (Puppet agent, Chef Infra Client, salt-minion).
- **Agentless** = không cài agent riêng của Ansible trên máy đích. Ansible chỉ cài trên **Control Node**, khi cần thì kết nối tới **Managed Node** qua kênh có sẵn (**SSH** với Linux, WinRM/SSH với Windows), làm xong thì rút ra.

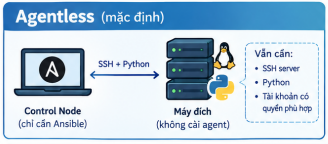
Mô hình Agent (Puppet, Chef)



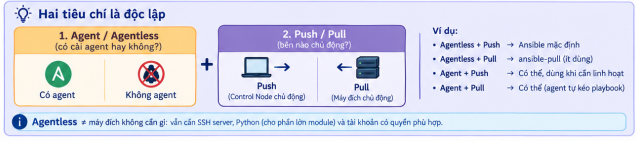
Mô hình Agentless (Ansible)



Có agent vs Agentless



Tiêu chí	Agentless	Agent
Cài trên máy đích	Không (chỉ cần SSH, Python có sẵn)	Có (cài agent + cập nhật agent)
Hạ tầng phụ	Không cần server trung tâm, không cần CA/key	Cần server trung tâm, CA/key cho từng agent
Chỉ cần Control Node	Có	Không (cần agent trên máy đích)
Tài nguyên máy đích khi rảnh	Không bị chiếm (chỉ khi chạy playbook)	Bị chiếm bởi agent (dù không chạy task)
Agent chạy nền	Không	Có (luôn chạy nền)
Tự sửa drift	Không (chỉ khi chạy lại playbook hoặc lập lịch)	Có (agent chạy định kỳ)
Quy mô rất lớn	Tốt (máy tự kéo cấu hình) Control Node chịu tải kết nối song song	Tốt (agent tự xử lý trên máy đích) Control Node nhẹ hơn



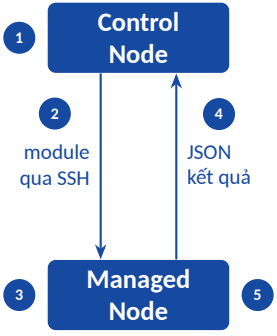
Yêu cầu của Control Node và Managed Node

	Control Node	Managed Node (Linux)
Ansible	Cài `ansible` / `ansible-core`	Không cần
Python	Phiên bản được bản Ansible hỗ trợ	Cần cho phần lớn module (không bắt buộc cùng phiên bản với Control Node)
SSH	SSH client	SSH server đang chạy, mở cổng TCP 22
Khác	Inventory, playbook, private key	Tài khoản có shell; quyền `sudo` nếu task cần root

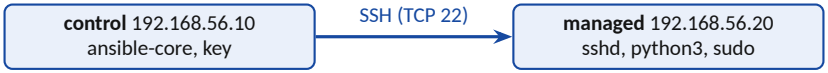
- Control Node phải là **Linux/macOS** (Windows thì dùng WSL); Managed Node có thể là Windows.
- Module `raw` chạy được cả khi máy đích chưa có Python → dùng để cài Python lần đầu.

Một task chạy như thế nào?

1. Control node đọc inventory, playbook, biến → đóng gói module + tham số.
2. SSH vào managed node, tạo thư mục tạm, chuyển mã module sang.
3. Managed node chạy module bằng Python (dùng `become` nếu cần quyền root).
4. Module trả **JSON** → Ansible báo `ok` / `changed` / `failed`.
5. Xoá file tạm; kết quả task (gói, file cấu hình) vẫn giữ.



Lab & SSH key



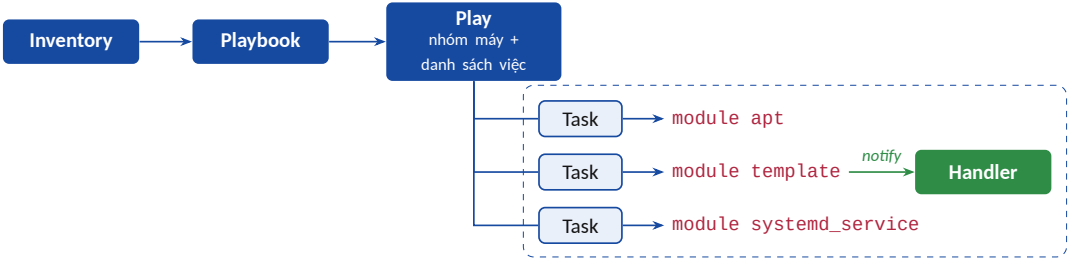

```
ssh-keygen -t ed25519 -f ~/.ssh/ansible_lab
ssh-copy-id -i ~/.ssh/ansible_lab.pub student@192.168.56.20
ansible web -m ansible.builtin.ping
# "pong" = SSH + Python OK (không phải ICMP)
```

```
# ansible.cfg
[defaults]
inventory = inventory.ini
remote_user = student
private_key_file = ~/.ssh/ansible_lab
```

- Private key **không bao giờ rời control node**; public key nằm trong `authorized_keys` của máy đích.
- `become: true` = chạy bằng `sudo`; **không tự cấp quyền** sudo. `-K` hỏi mật khẩu sudo.

01	Bài toán & sự ra đời	02	Kiến trúc Agentless
03	Các thành phần cơ bản	04	Idempotence
05	Module chuyên dụng vs Shell	06	Ansible Vault
07	Demo	08	Tổng kết & Hỏi đáp

Bức tranh tổng thể



Role = đóng gói tasks / handlers / templates / vars thành thư mục chuẩn

Inventory

```
# inventory.ini
[web]
managed ansible_host=192.168.56.20
```

- Inventory = **danh sách máy** Ansible quản lý, chia thành **nhóm**.
- Nhóm mặc định `all`; biến tách ra `group_vars/` và `host_vars/`.
- **Cụ thể hơn thắng**: `host_vars` > `group_vars/<nhóm>` > `group_vars/all`.

Playbook & Task

```yaml
- name: Cài nginx
  hosts: web
  become: true
  tasks:
    - name: Đảm bảo nginx đã cài
      ansible.builtin.apt:
        name: nginx
        state: present
```

- **Play**: `hosts`, `become`, `vars`, `tasks` (chạy từ trên xuống), `handlers`.
- **Task** = `name` + module + tham số; hay kèm `when`, `loop`, `register`, `notify`.
- **YAML**: thụt lề bằng **space**, `key: value`, list bắt đầu `-` .
- **Ad-hoc** (`ansible web -m ping`) = 1 lệnh dùng ngay; **playbook** = lưu Git, chạy lại được.

Role

```
roles/nginx/
├── tasks/main.yml      # task chính
├── handlers/main.yml   # handler
├── templates/          # file .j2
├── files/              # file tĩnh
├── defaults/main.yml   # biến mặc định – dễ ghi đè
└── vars/main.yml       # biến cố định của role
```

- Role **không tự chạy**; playbook gọi bằng `roles: [nginx]`.
- Ưu tiên biến:
  `defaults` < `group_vars` < `host_vars` < `-e`
  (extra vars **luôn thắng**).

Template Jinja2

- `{{ biến }}` in giá trị · `{% if %}` / `{% for %}` điều khiển · `{# #}` comment.
- Filter hay dùng: `default(80)`, `join(' ')`.
- Render **trên control node** rồi mới đẩy file sang máy đích.

	`copy`	`template`
Nội dung	Nguyên văn	Render biến rồi đẩy
Dùng cho	File tĩnh	File đổi theo host/môi trường

```
server {
    listen {{ nginx_port | default(80) }};
    server_name {{ server_name }};
{% for loc in locations %}
    location {{ loc.path }} {
        proxy_pass {{ loc.backend }};
    }
{% endfor %}
}
```

Handler

- Chỉ chạy khi task `notify` báo **`changed`** — config không đổi thì không restart.
- Chạy **cuối play**, **1 lần mỗi host** dù được notify nhiều lần.
- Tên trong `notify` phải **khớp** tên handler.

> Đề yêu cầu *restart*; thực tế nên dùng `state: reloaded` (không downtime) khi chỉ đổi config.

```yaml
- name: Render nginx.conf
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    validate: nginx -t -c %s
  notify: Restart nginx

# handlers/main.yml
- name: Restart nginx
  ansible.builtin.systemd_service:
    name: nginx
    state: restarted
```

01	Bài toán & sự ra đời	02	Kiến trúc Agentless
03	Các thành phần cơ bản	04	Idempotence
05	Module chuyên dụng vs Shell	06	Ansible Vault
07	Demo	08	Tổng kết & Hỏi đáp

Idempotent là gì?

- Chạy **1 lần hay 100 lần** → máy luôn ở **cùng trạng thái mong muốn**, không tác dụng phụ.
- Ví dụ: bấm **nút tầng 5** thang máy (idempotent) vs **nút toggle** đèn (không).

Idempotent	Không idempotent
`mkdir -p /data`	`mkdir /data` (lần 2 lỗi)
`echo X > file`	`echo X >> file` (nhân bản dòng)

Khai báo vs Mệnh lệnh

- Shell script: *"làm thế nào"*.
  Ansible: *"kết quả cuối là gì"*.
- Module: **kiểm tra hiện trạng** → khớp thì `ok`, lệch thì sửa vừa đủ → `changed`.

PLAY RECAP	Ý nghĩa
`ok`	Thành công (bao gồm cả `changed`)
`changed`	Đã thay đổi máy đích
`failed`	Lỗi, dừng play trên host đó
`skipped`	`when` trả về false

Xem trước: `--check` & `--diff`

- `--check`: **mô phỏng**, không đổi gì trên máy đích.
- `--diff`: hiện **dòng thêm/bớt** trong file.
- Giới hạn: `shell`/`command` bị **skip** trong check mode (trừ khi có `creates`/`removes`).

> `ansible-playbook site.yml --check --diff` — xem trước thay đổi trước khi áp dụng thật.

01	Bài toán & sự ra đời	02	Kiến trúc Agentless
03	Các thành phần cơ bản	04	Idempotence
05	Module chuyên dụng vs Shell	06	Ansible Vault
07	Demo	08	Tổng kết & Hỏi đáp

command vs shell

	command	shell
Chạy qua `/bin/sh`	Không	Có
Pipe, redirect, `$VAR`	Không	Có
Rủi ro shell injection	**Thấp**	**Cao**

- **Bẫy**: `command: echo data >> /tmp/x` → **không tạo file** (`>>` thành đối số) nhưng vẫn báo `changed`.
- `raw` (không cần Python, dùng bootstrap) và `script` (chạy script có sẵn) chỉ dùng trường hợp đặc biệt.

Vì sao module chuyên dụng tốt hơn?

- `shell`/`command` là **“hộp đen”**: Ansible chỉ thấy `rc`, `stdout` → **luôn báo changed**.
- Module chuyên dụng **kiểm tra trước khi làm**:
  - `copy`: so checksum, giống thì `ok`.
  - `lineinfile`: dòng đã có thì `ok`, chưa có thì thêm.
  - `systemd_service`: `state: started` chỉ khởi động khi đang dừng.
- Kèm theo: hỗ trợ `--check`/`--diff`, trả dữ liệu có cấu trúc, an toàn hơn, dễ đọc.

Khi buộc phải dùng shell

```yaml
- ansible.builtin.shell: tar -xzf app.tar.gz -C /opt/app
  args:
    creates: /opt/app/README     # đã có file này thì bỏ qua

- ansible.builtin.command: uname -r
  changed_when: false            # lệnh chỉ đọc
```

- `creates` / `removes`: cho Ansible biết khi nào lệnh **đã làm rồi** → bỏ qua, giữ idempotent.
- `changed_when: false`: lệnh chỉ đọc thì **không báo changed**.

01	Bài toán & sự ra đời	02	Kiến trúc Agentless
03	Các thành phần cơ bản	04	Idempotence
05	Module chuyên dụng vs Shell	**06**	**Ansible Vault**
07	Demo	08	Tổng kết & Hỏi đáp

Vault là gì?

- Secret để **plain text** trong Git sẽ **lộ vĩnh viễn** (Git nhớ lịch sử).
- Vault mã hoá **AES256** $\rightarrow$ secret vẫn nằm trong Git nhưng đã mã hoá.
- Lệnh: `create`, `encrypt`, `view`, `edit`, `decrypt`, `rekey`.
- Chạy: `--ask-vault-pass` (chạy tay) hoặc `--vault-password-file` (tự động; file `chmod 600` + `.gitignore`).

Cách tổ chức

```
group_vars/web/
├── vars.yml    # công khai:  app_secret: "{{ vault_app_secret }}"
└── vault.yml   # ĐÃ MÃ HOÁ:  vault_app_secret: "..."
```

- Tiền tố `vault_` → grep được tên biến, biết ngay giá trị thật nằm trong vault.
- Task dùng secret thêm `no_log: true` để **không in secret** ra màn hình.

01	Bài toán & sự ra đời	02	Kiến trúc Agentless
03	Các thành phần cơ bản	04	Idempotence
05	Module chuyên dụng vs Shell	06	Ansible Vault
07	**Demo**	08	Tổng kết & Hỏi đáp

Demo 1 + 2 — Role Nginx, Template & Handler

1. `ansible web -m ping` → chứng minh SSH key hoạt động.
2. `ansible-playbook site.yml` → role `nginx`: cài gói, render template, `started + enabled`.
3. Chạy lại → `changed=0`, **handler không chạy**.
4. Đổi biến (vd `nginx_port`) → template `changed` → **handler restart nginx**.
5. `curl http://192.168.56.20:<port>` kiểm chứng.

Demo 3 — Ansible Vault

1. `ansible-vault create group_vars/web/vault.yml` → `cat` thấy `$ANSIBLE_VAULT;1.1;AES256`.
2. Chạy playbook **thiếu mật khẩu** → lỗi; thêm `--ask-vault-pass` → chạy được.
3. Trên managed node, file secret được tạo với quyền `0600`; output không lộ secret nhờ `no_log`.

Demo 4 — Sự cố & Idempotency

1. Task `shell: echo "data" >> /file.txt`, chạy 3 lần → `cat /file.txt` có **3 dòng `data`**, lần nào cũng `changed`.
2. Xoá file, sửa task thành `lineinfile: path=/file.txt line=data create=true`.
3. Chạy 3 lần → lần 1 `changed=1`, lần 2 và 3 **`changed=0`**, file chỉ có 1 dòng.

> **Lưu ý:** phải xoá file trước bước 2, nếu không `lineinfile` thấy dòng đã có và báo `ok` ngay từ lần đầu. Ghi vào `/` cần `become: true`.

01	Bài toán & sự ra đời	02	Kiến trúc Agentless
03	Các thành phần cơ bản	04	Idempotence
05	Module chuyên dụng vs Shell	06	Ansible Vault
07	Demo	08	Tổng kết & Hỏi đáp

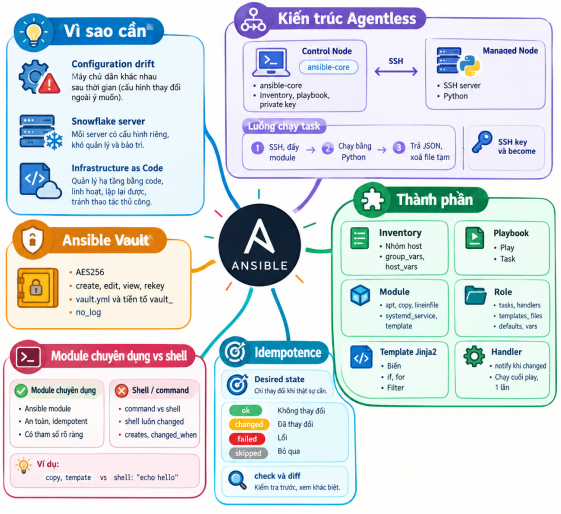
Tổng kết & Hỏi đáp

> **Agentless** qua SSH · **khai báo** trạng thái · **idempotent** · ưu tiên **module chuyên dụng**.

Câu hỏi hay gặp:

1. **Ansible khác shell script?** → Khai báo trạng thái, idempotent, chạy nhiều máy, không agent.
2. **`command` khác `shell`?** → `shell` qua `/bin/sh`; `command` an toàn hơn.
3. **Handler chạy khi nào?** → Cuối play, khi task notify báo `changed`, mỗi host 1 lần.
4. **`defaults` hay `vars`?** → Cho người dùng chỉnh → `defaults`; cố định → `vars`.

Mindmap kiến thức Ansible



fit@hcmus
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG - HCM
KHOA CÔNG NGHỆ THÔNG TIN

Cảm ơn thầy và các bạn đã lắng nghe!

Slide dự phòng

- **Handler nâng cao**: `listen` (gom handler theo topic), `meta: flush_handlers` (chạy ngay), `force_handlers` (chạy dù task sau lỗi).
- `reloaded` (HUP, không downtime) vs `restarted` (dừng rồi chạy lại).
- `encrypt_string`: mã hoá từng giá trị, tên biến vẫn đọc được.